In [1]:
```
# 도미 및 빙어 길이와 무게 데이터를 각각 리스트로 저장한다.

bream_length = [
    25.4, 26.3, 26.5, 29.0, 29.0, 29.7, 29.7,
    30.0, 30.0, 30.7, 31.0, 31.0, 31.5, 32.0,
    32.0, 32.0, 33.0, 33.0, 33.5, 33.5, 34.0,
    34.0, 34.5, 35.0, 35.0, 35.0, 35.0, 36.0,
    36.0, 37.0, 38.5, 38.5, 39.5, 41.0, 41.0
]

bream_weight = [
    242.0, 290.0, 340.0, 363.0, 430.0, 450.0, 500.0,
    390.0, 450.0, 500.0, 475.0, 500.0, 500.0, 340.0,
    600.0, 600.0, 700.0, 700.0, 610.0, 650.0, 575.0,
    685.0, 620.0, 680.0, 700.0, 725.0, 720.0, 714.0,
    850.0, 1000.0, 920.0, 955.0, 925.0, 975.0, 950.0
]

smelt_length = [9.8, 10.5, 10.6, 11.0, 11.2, 11.3, 11.8,
    11.8, 12.0, 12.2, 12.4, 13.0, 14.3, 15.0
]

smelt_weight = [
    6.7, 7.5, 7.0, 9.7, 9.8, 8.7, 10.0,
    9.9, 9.8, 12.2, 13.4, 12.2, 19.7, 19.9
]
```

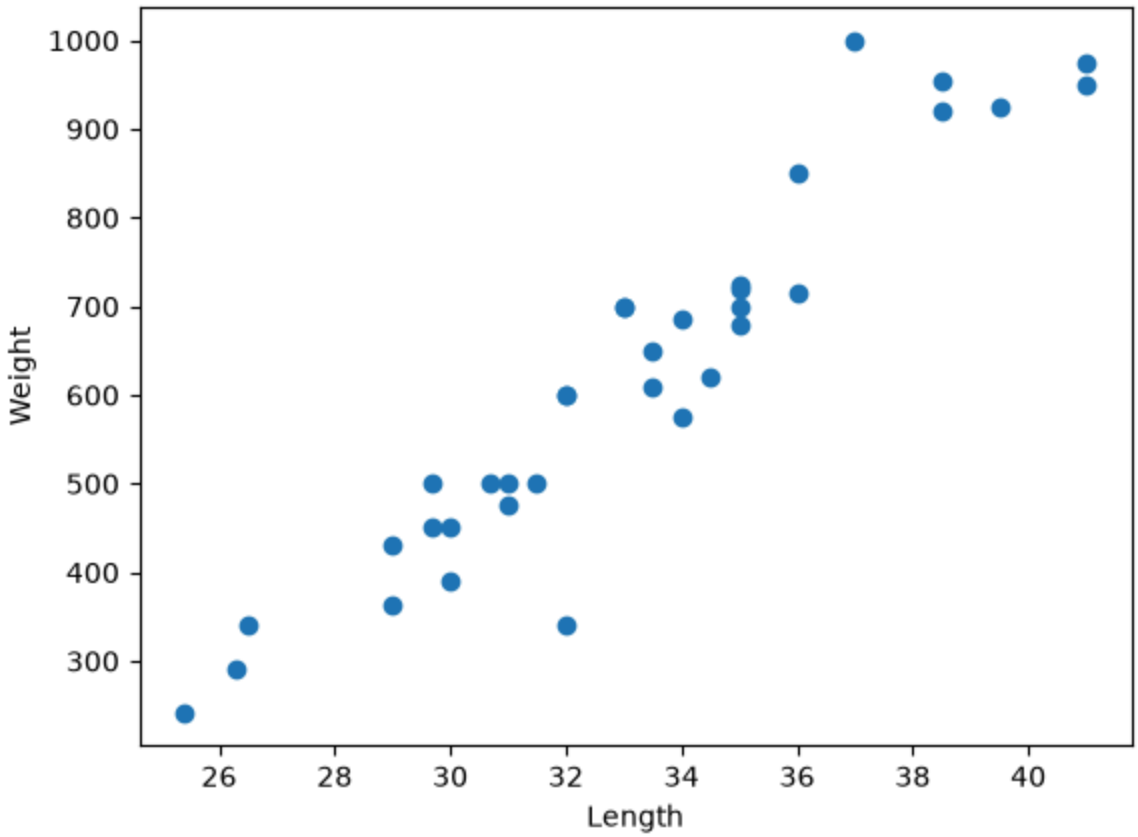
In [2]:
```
# matplotlib pyplot을 plt라는 별칭으로 불러온다.
# 도미의 길이를 x축, 무게를 y축으로 하여 산점도를 그린다.

import matplotlib.pyplot as plt

plt.scatter(bream_length, bream_weight)
plt.xlabel("Length")
plt.ylabel("Weight")

plt.show()

# ── 중간고사 학습 주석 (원본 코드 수정 없음) ──
# plt.scatter(x, y): 실제 데이터/예측 데이터를 산점도로 나타낸다.
# plt.show(): 지금까지 만든 그래프를 출력한다.
```



In [3]:
```
# 도미와 빙어의 길이 데이터를 하나의 리스트로 합친다.
# 도미와 빙어의 무게 데이터를 하나의 리스트로 합친다.

length = bream_length + smelt_length
weight = bream_weight + smelt_weight

print (length), print (weight)
print (len(length))
print (len(weight))
```

```
[25.4, 26.3, 26.5, 29.0, 29.0, 29.7, 29.7, 30.0, 30.0, 30.7, 31.0, 31.0, 31.
5, 32.0, 32.0, 32.0, 33.0, 33.0, 33.5, 33.5, 34.0, 34.0, 34.5, 35.0, 35.0, 3
5.0, 35.0, 36.0, 36.0, 37.0, 38.5, 38.5, 39.5, 41.0, 41.0, 9.8, 10.5, 10.6,
11.0, 11.2, 11.3, 11.8, 11.8, 12.0, 12.2, 12.4, 13.0, 14.3, 15.0]
[242.0, 290.0, 340.0, 363.0, 430.0, 450.0, 500.0, 390.0, 450.0, 500.0, 475.
0, 500.0, 500.0, 340.0, 600.0, 600.0, 700.0, 700.0, 610.0, 650.0, 575.0, 68
5.0, 620.0, 680.0, 700.0, 725.0, 720.0, 714.0, 850.0, 1000.0, 920.0, 955.0,
925.0, 975.0, 950.0, 6.7, 7.5, 7.0, 9.7, 9.8, 8.7, 10.0, 9.9, 9.8, 12.2, 13.
4, 12.2, 19.7, 19.9]
49
49
```

In [4]:
```
fish_data = [[L,W] for L, W in zip(length, weight)]
print (fish_data)

fish_target = [1]*35 + [0]*14
print(fish_target)

# ── 중간고사 학습 주석 (원본 코드 수정 없음) ──
# zip: 같은 위치에 있는 길이·무게를 짝지어 입력 특성 쌍을 만든다.
```

```
[[25.4, 242.0], [26.3, 290.0], [26.5, 340.0], [29.0, 363.0], [29.0, 430.0],
[29.7, 450.0], [29.7, 500.0], [30.0, 390.0], [30.0, 450.0], [30.7, 500.0],
[31.0, 475.0], [31.0, 500.0], [31.5, 500.0], [32.0, 340.0], [32.0, 600.0],
[32.0, 600.0], [33.0, 700.0], [33.0, 700.0], [33.5, 610.0], [33.5, 650.0],
[34.0, 575.0], [34.0, 685.0], [34.5, 620.0], [35.0, 680.0], [35.0, 700.0],
[35.0, 725.0], [35.0, 720.0], [36.0, 714.0], [36.0, 850.0], [37.0, 1000.0],
[38.5, 920.0], [38.5, 955.0], [39.5, 925.0], [41.0, 975.0], [41.0, 950.0],
[9.8, 6.7], [10.5, 7.5], [10.6, 7.0], [11.0, 9.7], [11.2, 9.8], [11.3, 8.7],
[11.8, 10.0], [11.8, 9.9], [12.0, 9.8], [12.2, 12.2], [12.4, 13.4], [13.0, 1
2.2], [14.3, 19.7], [15.0, 19.9]]
[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

In [5]:
```
# 훈련데이터와 테스트데이터를 나누어 학습한다
train_input = fish_data[:35] # 훈련데이터 (인덱스 0~34, 편향 분할 예제)
train_target = fish_target[:35]
test_input = fish_data[35:]
test_target = fish_target[35:]
print(len(train_input), len(train_target))
print(len(test_input), len(test_target))
# [오류 수정] 훈련 데이터 인덱스 범위 설명을 0~34로 수정했습니다. 편향 분할 시연 코드는 유지
```

```
35 35
14 14
```

In [6]:
```
# 학습하고 점수 확인
# 문제점 : 학습데이터에는 도미만, 테스트 데이터는 빙어만
# 공부는 도미로 했지만 테스트는 빙어로 한 것

from sklearn.neighbors import KNeighborsClassifier

kn = KNeighborsClassifier()
kn = kn.fit(train_input, train_target)

kn.score(test_input, test_target)

# ── 중간고사 학습 주석 (원본 코드 수정 없음) ──
# KNeighborsClassifier: 가까운 k개 이웃의 클래스로 분류하는 모델이며 기본 n_neighbors
# fit(X, y): 입력 데이터와 정답을 바탕으로 모델을 훈련한다.
# score(X, y): 분류에서는 정확도, 회귀에서는 결정계수 R²를 반환한다.
```

Out[6]:
```
0.0
```

In [7]:
```
#ㅜNumpy 이용하기

import numpy as np
```

```
input_arr = np.array(fish_data)
target_arr = np.array(fish_target)

np.random.seed(42)

index = np.arange(49)
np.random.shuffle(index)
#데이터 자체를 섞기는 힘드므로 인덱스 번호를 섞는다

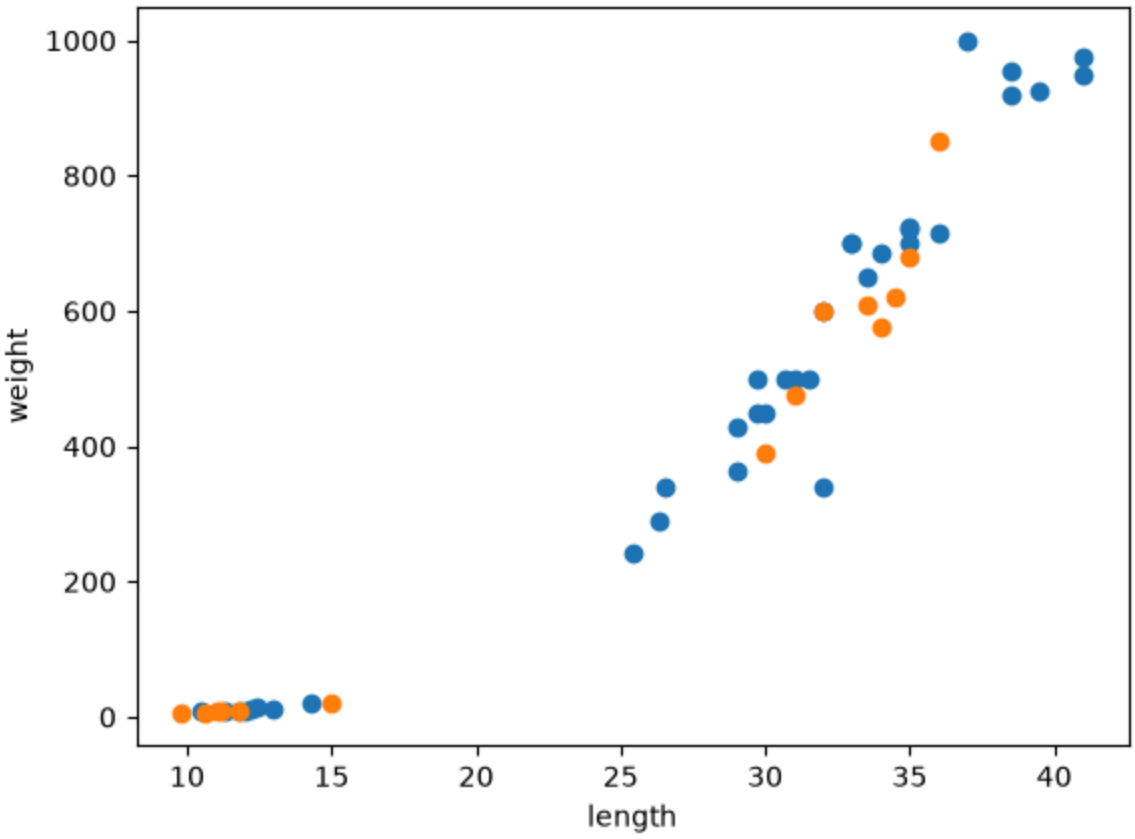
# —— 중간고사 학습 주석 (원본 코드 수정 없음) ——
# np.arange(start, stop): start부터 stop 직전까지 일정 간격의 값을 만든다.
# np.array: 파이썬 리스트 등을 NumPy 배열로 변환한다.
# np.random.shuffle: 인덱스 등의 배열을 제자리에서 무작위로 섞는다.
# np.random.seed: 난수 초기값을 고정한다.
```

In [8]:

```
train_input = input_arr[index[:35]]
train_target = target_arr[index[:35]]
test_input = input_arr[index[35:]]
test_target = target_arr[index[35:]]

import matplotlib.pyplot as plt
plt.scatter(train_input[:,0], train_input[:,1])
plt.scatter(test_input[:,0], test_input[:,1])
plt.xlabel('length')
plt.ylabel('weight')
plt.show()

# —— 중간고사 학습 주석 (수정본) ——
# [오류 수정] 인덱스로 학습/테스트 입력과 정답을 일치시켜 분리하고, scatter 함수명을 교정했
```



온라인 2주차에서 보충 — 섞은 데이터로 다시 학습하고 결과 확인

In [9]:
```
# [온라인 2주차 보충] 위에서 섞은 인덱스를 이용한 데이터로 모델을 다시 훈련한다.
# 중요: 앞에서 kn은 도미 데이터만 학습했으므로, 섞은 뒤에는 fit()을 다시 해야 한다.
kn.fit(train_input, train_target)

# score()는 분류 문제에서 정답을 맞힌 비율(정확도)을 돌려준다.
print('train score:', kn.score(train_input, train_target))
print('test score:', kn.score(test_input, test_target))

# predict()로 테스트 세트의 실제 분류값을 예측한다.
print('test target:', test_target)
print('test prediction:', kn.predict(test_input))

# shape: 입력 특성은 (샘플 수, 특성 수), 정답은 (샘플 수,) 형태이다.
print('train shapes:', train_input.shape, train_target.shape)
print('test shapes:', test_input.shape, test_target.shape)
```

```
train score: 1.0
test score: 1.0
test target: [0 0 1 0 1 1 1 0 1 1 0 1 1 0]
test prediction: [0 0 1 0 1 1 1 0 1 1 0 1 1 0]
train shapes: (35, 2) (35,)
test shapes: (14, 2) (14,)
```

In [ ]: